
Suggested Teaching Guidelines for

Practical Machine Learning

PGCP-BDA February 2026

Duration: 60 Hours Theory + 80 Hours Lab + 10 Hours Self Learning

Objective: Practicing Machine Learning Algorithms

Prerequisites: Good knowledge of Python Programming and Statistics

Evaluation method:

Theory exam– 40% weightage
Lab exam – 40% weightage
Internal exam– 20% weightage

List of Books / Other training material

Courseware:

Machine Learning, Saikat Dutt, Amit Kumar Das, S Chandramouli/ Pearson

Reference Book:

Machine Learning using Python , Manaranjan Pradhan , U Dinesh Kumar, Wiley

Session 1: (2T+2L+1SL)

Theory and Lab:

- Fundamentals of information theory
- Introduction to machine learning
- Algorithm types of Machine learning
- Probably Approximately Correct (PAC) Learning
- Uses of Machine learning
- Evaluating ML techniques

Self Learning:

- Explore how information theory underpins learning, generalization, and uncertainty handling
- Study why machine learning is framed as a learning problem rather than a programming problem
- Examine different learning paradigms and their suitability for real-world problems
- Analyze why evaluation is essential before claiming a model is “good”

Suggested Teaching Guidelines for

Practical Machine Learning

PGCP-BDA February 2026

Session 2: (2T+2L+1SL)

Theory and Lab:

- Bias complexity trade off
- Vapnik-Chervonenkis (VC) Dimension
- Non-uniform learnability (Structural risk minimization, Occam's Razor and No Free Lunch Theorem)
- Regularization and Stability

Self Learning:

- Study bias–complexity trade-off as a central challenge in model design
- Explore VC dimension as a measure of model capacity rather than algorithm strength
- Examine and document why no single algorithm performs best across all problems (No Free Lunch)

Session 3: (2T + 2L)

Theory:

- Model Selection and Validation
- Introduction to Scikit Learn
- Performing ML using Scikit Learn

Lab:

- Explore scikit learn Library.
- Explore Datasets Online (can refer Kaggle, UCI ML, etc.)
 - Load dataset in google colab.
 - Print first five values and last five values in dataset.
 - check correlation between fields present in dataset

Session 4 & 5: (4T+4L+2SL)

Theory:

- Clustering
- Hierarchical Clustering & K means
- Distance Measure and Data Preparation – Scaling & Weighting
- Evaluation and Profiling of Clusters
- Hierarchical Clustering
- Clustering Case Study
- Principal Component analysis: PCA, Kernel PCA
- Random Projections

Suggested Teaching Guidelines for

Practical Machine Learning

PGCP-BDA February 2026

Lab:

Download “mall_customers.csv” dataset from Kaggle.

- (a) Form n no. of clusters according to your observation.
- (b) Get wss value for each cluster.
- (c) find best K value

Self Learning:

- Examine and document why clustering is inherently subjective and exploratory
- Study how distance measures and scaling influence cluster formation
- Reflect on PCA and random projections as tools for representation, not compression alone
- Analyze how clustering evaluation differs from supervised learning evaluation

Session 6 & 7: (4T+6L)

Theory:

- Evaluation metrics: Accuracy, F1-score, ROC AUC, Log Loss
- Evaluation metrics: MAE, RMSE, R2 score
- Decision Trees
- Classification and Regression Trees
- Random forest, Gradient boosting Machines, Model Stacking
- CAT Boost & Light GBM
- XG Boost

Lab:

- Implement Random Forest, SVM, Logistic regression classification algorithm
- Check for classification report, f1 score for all three algorithms.

Session 8 & 9: (4T + 4L)

Theory:

- Bayesian analysis and Naïve bayes classifier
- Assigning probabilities and calculating results
- Linear Discriminant Analysis
- K-Nearest Neighbors Algorithm

Suggested Teaching Guidelines for

Practical Machine Learning

PGCP-BDA February 2026

Lab:

- Implement K-Nearest Neighbors Algorithm

Session 10 & 11: (4T+6L+1SL)

Theory:

- Linear Regression
- Logistic Regression
- Polynomial Regression
- Ridge Regression
- Lasso Regression
- Elastic Net Regression

Lab:

- Download Dataset, perform linear, Ridge, Lasso, Polynomial regression and check for MAE, MSE, RMSE, F1 score and explain with conclusion.

Self Learning:

- Study regression as a modeling tool rather than a curve-fitting exercise
- Reflect on why regularization is essential in high-dimensional data
- Compare different regression techniques from a bias–variance perspective

Session 12: (2T+4L+1SL)

Theory:

- Support Vector Machines
- Basic classification principle of SVM
- Linear and Nonlinear classification (Polynomial and Radial)

Lab:

- Download Air Quality Dataset from Kaggle Predict Air Quality Index using Linear regression and classify it into five categories using SVM (i.e. Very good, good, moderate, poor, worst)

Self Learning:

- Visualize margin maximization as a geometric learning principle
- Study kernel methods as implicit feature transformation techniques
- Examine and document trade-offs between linear and non-linear decision boundaries

Suggested Teaching Guidelines for

Practical Machine Learning

PGCP-BDA February 2026

Session 13 & 14: (4T + 4L)

Theory:

- Moving average, Exponential Smoothing, Holt's Trend Methods, Holt-Winters'
- Methods for seasonality
- Autocorrelation (ACF & PACF), Auto-regression, Auto-regressive Models, Moving Average Models
- ARMA & ARIMA

Lab:

- What is Auto correlation, explain its purpose Also download one data set and calculate Auto correlation.
- Explain ARMA and ARIMA model, what is purpose of this models in time series and Explain difference between them.

Session 15 & 16: (2T+4L)

Theory and Lab:

- Recommendation Systems
 - Data Collection & Storage, Data Filtering
 - Collaborative Filtering
 - Factorization Methods
 - Evaluation Metrics: Recall, Precision, RMSE, Mean Reciprocal Rank, MAP at K, NDCG

Session 17, 18 & 19: (4T+4L)

- Introduction to Deep Learning
- Introduction to Tensor flow and Keras

Lab:

- Explore Tensor Flow and Keras Libraries

Session 18 & 19: (4T+6L+1SL)

Theory:

- Introduction to Auto-encoders
- Neural Network and its applications
- Single layer neural Network
- Activation Functions: Sigmoid, Hyperbolic Tangent, ReLu
- Overview of Back propagation of errors

Suggested Teaching Guidelines for

Practical Machine Learning

PGCP-BDA February 2026

Lab:

- Implement Different Activation functions on datasets in Jupyter Notebook.

Self Learning:

- Explore why representation learning is central to modern deep learning and how auto-encoders differ from predictive models
- Study the role of single-layer neural networks as building blocks for deeper architectures
- Examine and document why non-linear activation functions are essential for learning complex patterns
- Analyze backpropagation as an error-correction mechanism rather than a mathematical procedure

Session 20: (2T+2L+2SL)

Theory and Lab:

Deep Learning Essentials

- Early Stopping for Preventing Overfitting
 - Dropout
 - L1 and L2 Regularization
- Update of weights with single training set element

Self Learning:

- Examine and document why deep neural networks tend to overfit when model capacity exceeds data complexity
- Study regularization techniques as learning constraints rather than post-training fixes
- Research early stopping as a practical balance between underfitting and overfitting
- Analyze weight updates using single training examples to understand stochastic learning behavior

Session 21 & 22: (4T+6L+1SL)

Theory:

- Batch Training, Mini-batch Training, Stochastic Gradient Descent
- Training Methods for Neural Network (High-Level Overviews only) / Optimizers
 - Classic Back propagation
 - Momentum Back propagation
 - ADAM

Suggested Teaching Guidelines for

Practical Machine Learning

PGCP-BDA February 2026

Lab:

- Implement Different optimizers
- Implement Gradient Problems

Self Learning:

- Compare batch, mini-batch, and stochastic training from a convergence and efficiency perspective
- Study optimization algorithms as navigation strategies on an error surface
- Reflect on why adaptive optimizers such as ADAM dominate modern deep learning practice
- Analyze trade-offs between faster convergence and long-term generalization

Session 23 & 24: (4T + 6L)

Theory:

Convolutional Neural Network using PyTorch

- Introduction to PyTorch Framework
- Pytorch vs Tensor flow
- Convolutional Concept
- Transfer Learning and Inception Network as one of its examples
- Data Augmentation
- Object Detection
- YOLO Algorithm (High-Level Overview)

Lab:

- Install PyTorch. Explore the documentation of PyTorch Library.
- Implement YOLO Algorithm.

Self Learning:

- Research and document why PyTorch is preferred for deep learning experimentation and research-oriented workflows
- Study convolution as a representation-learning mechanism rather than an image-processing trick
- Examine transfer learning as a strategy for leveraging prior knowledge in data-scarce environments
- Analyze why object detection is more complex than image classification and how YOLO reframes detection as a single-step prediction problem

Suggested Teaching Guidelines for

Practical Machine Learning

PGCP-BDA February 2026

Session 25 & 26: (4T + 6L)

Theory:

Recurrent Neural Network (RNN) using Pytorch

- RNN Concept
- Types of RNNs
- Vanishing gradients with RNNs
- Gated Recurrent Unit (GRU) - (High-Level Overview only)
- Long Short-Term Memory (LSTM) - (High-Level Overview only)

Lab:

- Implement RNN using PyTorch
- Implement LSTM and GRU

Case Studies:

- Time series example with LSTM

Self Learning:

- Research why sequence data cannot be modeled effectively using feedforward architectures
- Study temporal dependency and how information flows across time steps in RNNs
- Analyze the vanishing gradient problem as a limitation of naive recurrence
- Examine how gating mechanisms in LSTM and GRU address long-term dependency learning

Session 27: (2T + 4L)

Theory:

Generative AI: Introduction to Transformers

- Transformers and their importance
- A brief history of NLP and the rise of transformers (including the Attention Timeline)
- Transformers
 - Introduction to BERT (Bidirectional Encoder Representations from Transformers):
 - Pre-training objectives of BERT and its impact on NLP tasks
 - Fine-tuning BERT for various NLP applications (e.g., sentiment analysis, question answering)

Suggested Teaching Guidelines for

Practical Machine Learning

PGCP-BDA February 2026

- Understanding and Handling Text Data:
 - Text pre-processing techniques (e.g., tokenization, stemming, lemmatization)
 - Representing text data for machine learning models
 - Hands-on activity: Pre-processing text data and fine-tuning a pre-trained model for a specific task
- Applications of transformers across various domains (e.g., computer vision, speech recognition)

Lab:

- Hands-on activity: Exploring a pre-trained transformer model (e.g., using Google AI's Colab)

Session 28: (2T + 2L)

Theory and Lab:

- Core Concepts of AI architectures
- Understanding the Encoder-Decoder Architecture:
 - The role of encoders in processing input sequences
 - The role of decoders in generating output sequences
 - Encoder-decoder for tasks like machine translation and text summarization
- Attention Mechanisms:
 - Demystifying attention and its variants (e.g., self-attention, masked attention)
 - How attention helps models "focus" on relevant parts of the input
- Hands-on activity: Implementing a simple attention mechanism

Session 29 & 30: (4T + 6L)

Theory:

- Advanced Concepts & Applications
 - Introduction to Large Language Models (LLMs)
 - Capabilities and limitations of LLMs
 - Responsible development and use of LLMs
 - Reward Models and Alignment Strategies
 - Ensuring alignment between LLM goals and human values
 - Techniques for mitigating bias and promoting safety
- Practical Case Studies:
 - Exploring real-world applications of transformers and LLMs (e.g., chatbots, text generation, code completion)

Suggested Teaching Guidelines for

Practical Machine Learning

PGCP-BDA February 2026

- Deployment Considerations for LLMs:
 - Infrastructure requirements and challenges
 - Strategies for efficient deployment of LLMs in production environments

Lab:

- Exploring an LLM API or building a simple application using a pre-trained model